

A TDMA Algorithm for Dual-Hop Energy Harvesting Wireless Sensor Networks

Tanner D. Patterson 

ECCS Dept. Ohio Northern University
Ada, Ohio, USA
t-patterson.4@onu.edu

Robert J. Hayek 

ECE Dept. Northwestern University
Evanston, Illinois, USA
robert.hayek@u.northwestern.edu

Ahmed Ammar 

ECCS Dept. Ohio Northern University
Ada, Ohio, USA
a-ammar@onu.edu

Abstract—In this paper, we propose a time division multiple access (TDMA) algorithm and system for a dual-hop energy harvesting wireless sensor network. We assume that the network is divided into clusters, where each cluster has a cluster head and cluster members. A cluster head collects data from its respective cluster members and then transmits the data in one packet to the base station. The proposed algorithm first synchronizes the network and then enables each node to transmit over its designated time slot. The proposed algorithm allows each node to synchronize not only based on a synchronization packet from the base station, but also based on a data packet from another node that is within its communication range. Moreover, the proposed algorithm synchronizes clusters individually, i.e., if a cluster becomes out-of-sync, the algorithm only synchronizes that cluster while allowing the remaining clusters to continue normal operation. The proposed algorithm works also for single-hop networks, and achieves network synchronization without the need for acknowledgment packets, which reduces the average energy consumption and average delay by approximately 1.36 and 1.46 times, respectively, compared to the most recent single-hop algorithm.

Index Terms—Time Division Multiple Access (TDMA), wireless sensor networks, energy harvesting, synchronization, dual-hop.

I. INTRODUCTION

Wireless Sensor Networks (WSNs) have been employed in various fields [1], including environmental monitoring, intelligent transportation, building automation, military surveillance, etc. In recent developments, energy harvesting technology has been incorporated into WSNs to extend their lifespan and reduce costs associated with batteries replacement/recharging. However, the stochastic nature of energy harvesting processes imposes energy availability constraints on the system, necessitating the need for new WSNs protocols and algorithms that can handle these constraints.

Numerous protocols and algorithms have been proposed in the literature for energy harvesting WSNs [2]–[10]. Specifically, in [7]–[10], the authors propose TDMA algorithms for a single-hop energy harvesting WSN. The CEH-TDMA and AT-MAC algorithms, proposed in [7] and [8], respectively, follow a process of initially synchronizing sensor nodes and then periodically allocate time slots for them. However, these algorithms exhibit two significant drawbacks. Firstly, node synchronization occurs only once, namely at the onset of network operation, which is impractical due to clock drift issues [11], [12]. Secondly, time slots are allocated in each period based on information received from the nodes, requiring

sensor nodes to transmit information every period, leading to high energy consumption. The EH-TDMA algorithm presented in [9] addresses the clock drift problem by periodically synchronizing sensor nodes and mitigates energy consumption by employing a fixed time slot allocation in every period. A similar approach is adopted in [10]; however, the algorithm in [10] achieves a longer synchronization period, resulting in not only lower energy consumption, but also less delay.

In this paper, we propose a TDMA algorithm and system for a dual-hop energy harvesting WSN, comprising sensor nodes and a base station. We assume the network is divided into clusters, each has a cluster head and cluster members. Each cluster head collects data from its respective cluster members, aggregates the data in one packet, and subsequently transmits the packet to the base station. The proposed algorithm enables a cluster head to synchronize based on a synchronization packet from the base station, or a data packet from another cluster head, or a data packet from one of its respective cluster members. Similarly, a cluster member can synchronize based on a synchronization packet from the base station, or a data packet from its respective cluster head, or a data packet from one of its respective cluster members. In contrast to [9], the proposed algorithm necessitates a complete network resynchronization only when all nodes are out-of-energy or when a transmission overlap occurs between cluster heads. Notably, the proposed algorithm enables individual synchronization of clusters, i.e., if a cluster falls out-of-sync, the algorithm will exclusively resynchronize that cluster while allowing other clusters to continue normal operation. Moreover, the proposed algorithm works also for single-hop networks, and in contrast to [10], the proposed algorithm achieves network synchronization without the need for acknowledgment packets. To the best of our knowledge, this is the first work that proposes a TDMA algorithm for a dual-hop energy harvesting WSN.

We implemented the proposed algorithm on an Arduino-based WSN. The proposed algorithm was evaluated based on its average energy consumption and average delay. Similar to [10], average energy consumption is defined as the average energy consumed by a node and the base station due to synchronization and time slot allocation. Average delay is defined as the total time for synchronization and time slot allocation. Compared to the algorithm in [10], the proposed algorithm reduces the average energy consumption and average delay by approximately 1.36 and 1.46 times, respectively.

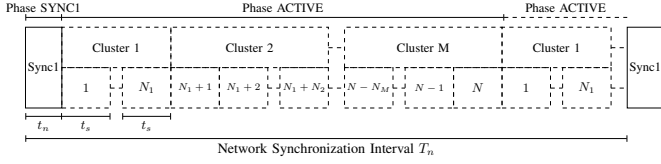


Fig. 1. Proposed TDMA system.

II. SYSTEM MODEL

We consider a dual-hop energy harvesting WSN, consisting of N sensor nodes and a base station. The base station coordinates the network synchronization, transmissions, and data aggregation. We assume that the network is partitioned into M clusters, where cluster $m \in \{1, 2, \dots, M\}$ has N_m sensor nodes. In each cluster, one of the nodes acts as a cluster head while the rest act as cluster members. Cluster heads can communicate directly with the base station, with other cluster heads, and with their respective cluster members. Cluster members can communicate with their respective cluster heads and cluster members, and can only receive from the base station. Cluster members transmit their data to their respective cluster head, which aggregates the data in a single packet and transmits it to the base station. Similar to [9] and [10], we assume that nodes always have data to transmit and that each node transmits its data in a unique time slot, incorporating its time slot number within the transmission packet.

We assume that only sensor nodes are energy harvesting and that nodes follow the Harvest-Store-Spend policy [2], [8], [10], [13], where sensor nodes harvest energy, store it in the battery, and then use it in a later time. We model the energy harvesting processes of the sensor nodes as discrete and independent random variables. Assuming that sensor nodes are equipped with rechargeable batteries of a large capacity and transmit with a constant power as in [2], [13], then the energy availability event E_i at node $i \in \{1, 2, \dots, N\}$ can be modeled as a binary random variable with probability p_i , where $E_i = 1$ represents the event that node i has sufficient energy while $E_i = 0$ represents the event that node i does not have sufficient energy, which is also called the out-of-energy event in this paper. As defined in [2], [13], p_i is the ratio of the energy harvesting rate to the energy depletion rate.

III. PROPOSED TDMA SYSTEM

The proposed TDMA system, as shown in Fig. 1, is comprised of two phases: *SYNC1* and *ACTIVE*. Phase *SYNC1* imposes a synchronization on the entire network. Phase *ACTIVE* immediately follows phases *SYNC1*, and during which nodes transmit their data packets, wait for allotted time slot, and synchronize when needed.

As shown in the figure, phase *ACTIVE* is divided into N time slots each with a length of t_s . The time slots are assigned to the nodes such that time slots from 1 to N_1 are assigned to the nodes of cluster 1, time slots from $N_1 + 1$ to $N_1 + N_2$ are assigned to the nodes of cluster 2, and so on. The cluster heads are assigned the last time slot of the time slots assigned to their clusters, i.e., cluster head m is assigned time slot $\sum_{i=1}^m N_i$. Phase *ACTIVE* will repeat until a network error

is detected at the base station. A network error is defined by either all nodes are out-of-energy or a transmission overlap between cluster heads, caused by clock drift between cluster heads. If a network error occurs, meaning the entire network is out-of-sync, the network will transition to phase *SYNC1*, where a synchronization on the entire network will be imposed, and then phase *ACTIVE* will begin, continuing the same process.

As illustrated in Fig. 2, there is another phase in the TDMA system, called *SYNC2*, and this phase occurs during Phase *ACTIVE*. Phase *SYNC2* is to impose a synchronization on a particular cluster if a cluster error is detected at the base station. A cluster error is defined by a transmission overlap between cluster members within the same cluster, caused by clock drift between them. If a cluster error occurs, meaning that the cluster members within that cluster have become out-of-sync, that cluster will transition to phase *SYNC2* to resynchronize while remaining in phase *ACTIVE*.

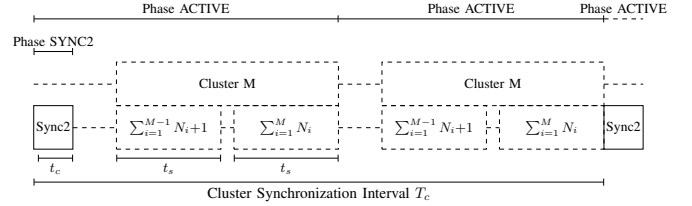


Fig. 2. Phase *SYNC2* within phase *ACTIVE*.

As shown in Fig. 1 and 2, the time between two *SYNC1* phases is called network synchronization interval and denoted by T_n while the time between two *SYNC2* phases is called cluster synchronization interval and denoted by T_c . We denote the length of phase *SYNC1*, the length of phase *SYNC2*, and the number of *ACTIVE* phases between network synchronizations by t_n , t_c , and P , respectively. Using t_n , t_c , and P , $T_n = t_n + P N t_s$ and $T_c = t_c + P N t_s$. P is random due to the randomness of clock drifts and energy availability at nodes, causing random T_n and T_c . Thus, the proposed algorithm is evaluated based on average energy consumption and average delay.

IV. PROPOSED ALGORITHM

This section describes the proposed TDMA algorithm that implements the TDMA system presented in Section III. The proposed algorithm comprises three algorithms: Algorithm 1, Algorithm 2, and Algorithm 3, described next.

A. Algorithm 1

Algorithm 1 is executed at the base station. As evident from the algorithm, the base station undergoes three phases: *SYNC1*, *ACTIVE*, and *SYNC2*. Upon initialization, the base station initiates phase *SYNC1*, where it first broadcasts a Sync1 packet and subsequently transitions to phase *ACTIVE*. In phase *ACTIVE*, the base station receives cluster heads packets, marks their timestamps, extracts timestamps of cluster members' packets from them, and then uses timestamps to detect errors. Section V-A. explains how the base station uses timestamps to detect errors. The base station remains in phase *ACTIVE* as long as no error is detected. If a network error is detected

Algorithm 1: Base Station Algorithm

```
1 begin
2   set phase to SYNC1;
3   switch phase do
4     case SYNC1 do
5       broadcast Sync1 packet;
6       set phase to ACTIVE;
7     end
8     case ACTIVE do
9       while no errors do
10        receive cluster heads packets and mark
11         their timestamps;
12        extract timestamps from cluster heads
13         packets;
14        use timestamps to detect errors;
15        if error detected then
16          if network error detected then
17            set phase to SYNC1;
18          else
19            set phase to SYNC2;
20          end
21        end
22      end
23    case SYNC2 do
24      update Sync2 packet header;
25      broadcast Sync2 packet;
26      set phase to ACTIVE;
27    end
28 end
```

(i.e., all nodes are out-of-sync), the base station will transition to phase *SYNC1* and broadcast a Sync1 packet, imposing a network resynchronization. If a cluster error is detected (i.e., cluster members in cluster m are out-of-sync), the base station will transition to phase *SYNC2* and broadcast a Sync2 packet, imposing cluster m to resynchronize. The header of Sync2 packet includes the cluster number m that needs to resynchronize and the current time slot $k \in \{1, 2, \dots, N\}$. m and k are updated every time the base station enters phase *SYNC2*. After transmitting Sync2 packet, the base station transitions to phase *ACTIVE* to repeat the same process.

B. Algorithm 2

Algorithm 2 is executed at cluster members. As evident from the algorithm, a cluster member undergoes four phases: *DEAD*, *SYNC*, *WAIT*, and *TRANSMIT*. Upon initialization, a cluster member initiates phase *DEAD* and remains in this phase until it harvests sufficient energy. Once the cluster member has sufficient energy, it promptly transitions to phase *SYNC*, waiting for a synchronization packet, which is either a Sync1 or Sync2 packet from the base station or a data packet from one of its cluster members or its cluster head. If a synchronization packet is received, the cluster member will synchronize based on that packet and then transition to phase

Algorithm 2: Cluster Member Algorithm

```
1 begin
2   set phase to DEAD;
3   switch phase do
4     case DEAD do
5       harvest energy;
6       if sufficient energy collected then
7         set phase to SYNC;
8       end
9     end
10    case SYNC do
11      wait for a synchronization packet;
12      if synchronization packet received then
13        synchronize using the packet;
14        set phase to WAIT;
15      end
16    end
17    case WAIT do
18      wait for designated time slot;
19      receive packets;
20      if Sync1 or Sync2 packet received then
21        set phase to SYNC;
22      end
23      if designated time slot then
24        set phase to TRANSMIT;
25      end
26    end
27    case TRANSMIT do
28      send packet;
29      if still has sufficient energy then
30        set phase to WAIT;
31      else
32        set phase to DEAD;
33      end
34    end
35  end
36 end
```

WAIT, where it will wait for its designated time slot. While waiting, if a Sync1 or Sync2 packet is received, the cluster member will transition back to phase *SYNC*. Once the designated time slot is reached, the cluster member will transition to phase *TRANSMIT*, where it transmits its data packet and subsequently checks its remaining energy. If sufficient energy remained, the cluster member will transition to phase *WAIT*, otherwise, it will transition to phase *DEAD*. From these two phases, the node will continue the same process. Section V-B discusses how a cluster member waits for its designated time slot and uses a synchronization packet to synchronize.

C. Algorithm 3

Algorithm 3 is executed at cluster heads, and is similar to Algorithm 2. However, in Algorithm 3, the synchronization packet can also be a packet from another cluster head. In addition, in Algorithm 3, the phases *SYNC*, *WAIT*, and *TRANSMIT* have more tasks.

Algorithm 3: Cluster Head Algorithm

```

1 begin
2   set phase to DEAD;
3   switch phase do
4     case DEAD do
5       harvest energy;
6       if sufficient energy collected then
7         set phase to SYNC;
8       end
9     end
10    case SYNC do
11      wait for a synchronization packet;
12      if synchronization packet received then
13        synchronize using the packet;
14        if a cluster member packet then
15          mark the packet timestamp;
16          store the packet and its timestamp;
17        end
18        set phase to WAIT;
19      end
20    end
21    case WAIT do
22      wait for designated time slot;
23      receive packets;
24      if Sync1 or Sync2 packet received then
25        set phase to SYNC;
26      end
27      if a cluster member packet then
28        mark the packet timestamp;
29        store the packet and its timestamp;
30      end
31      if designated time slot then
32        set phase to TRANSMIT;
33      end
34    end
35    case TRANSMIT do
36      combine all stored data in one packet;
37      transmit the packet;
38      if still has sufficient energy then
39        set phase to WAIT;
40      else
41        set phase to DEAD;
42      end
43    end
44  end
45 end

```

In phase *SYNC*, if the synchronization packet is a data packet from one of its respective cluster members, the cluster head marks the timestamp of that packet and stores it with that packet until its designated time slot is reached. In phase *WAIT*, if a data packet from one of its respective cluster members is received, the cluster head also marks the timestamp of that packet and stores it with that packet. In phase *TRANSMIT*, the cluster head combines all the stored data (cluster members' packets and their timestamps) in one packet, and transmits

that packet to the base station. As mentioned in IV-A, these timestamps are used at the base station to detect errors, which will be explained later in Section V-A. Finally, the transition between the states in Algorithm 3 is similar to Algorithm 2.

Note that in Algorithm 1, the base station transitions to phase *ACTIVE* immediately after transmitting Sync1 packet. In Algorithms 2 and 3, a cluster member and cluster head transition to phase *SYNC* immediately if an error is detected. These immediate transitions eliminate the needs for acknowledgement packets, which are part of the algorithm in [10].

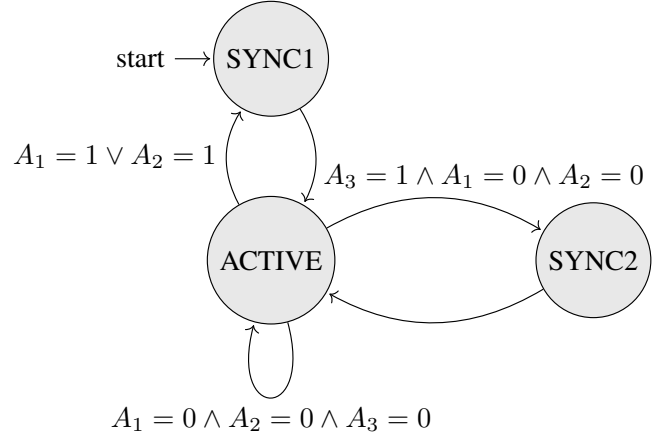


Fig. 3. State Diagram for Algorithm 1.

V. ALGORITHM IMPLEMENTATION

The proposed algorithms were implemented as Finite State Machines (FSMs) for their simplicity and speed in troubleshooting and coding. These FSMs are described next.

A. FSM for Algorithm 1

Fig. 3 illustrates the FSM for Algorithm 1. The FSM has three states: *SYNC1*, *ACTIVE*, and *SYNC2*, which implement phases: *SYNC1*, *ACTIVE*, and *SYNC2*, in Algorithm 1, respectively. At start-up, the base station initiates state *SYNC1*, where it broadcasts a Sync1 packet and immediately transitions to state *ACTIVE*. In state *ACTIVE*, the base station receives cluster heads packets, marks their timestamps, extracts timestamps of cluster members' packets from them, and then uses timestamps to detect network and cluster errors. A network error is either an out-of-energy error denoted by A_1 or a cluster heads transmission overlap error denoted by A_2 . A cluster error is a cluster members transmission overlap error denoted by A_3 . The base station remains in state *ACTIVE* until an error is detected. If a network error is detected, i.e., $A_1 = 1 \vee A_2 = 1$, the base station will immediately transition to state *SYNC1* to broadcast a Sync1 packet. If a cluster error is detected while no network error, i.e., $A_3 = 1 \wedge A_1 = 0 \wedge A_2 = 0$, the base station will immediately transition to state *SYNC2*, where it updates the m and k values in the Sync2 packet and then broadcasts it. After broadcasting a Sync1 or Sync2 packet, the base station will transition to state *ACTIVE*, continuing the same process.

The base station uses the timestamps to detect errors. The out-of-energy error is detected using the timestamp of the latest received cluster head packet τ_l and the current time at the base station τ_c . If $\tau_c - \tau_l \geq (h + 1)Nt_s$, where $h = 1$ if a single-hop and $h = 2$ if a dual-hop, the base station declares an out-of-energy error, i.e., $A_1 = 1$. The cluster heads transmission overlap error is detected using the timestamp τ_l and the timestamp of the previous cluster head packet τ_p . If $\tau_l - \tau_p \leq M_t$, where M_t is data packet transmission time duration, the base station declares a cluster heads transmission overlap error, i.e., $A_2 = 1$. The base station detects cluster members transmission overlap error by using the timestamps of the cluster members' packets, included in the cluster heads packets. If the time difference between any two consecutive timestamps is less than M_t , the base station declares a cluster members transmission overlap error, i.e., $A_3 = 1$.

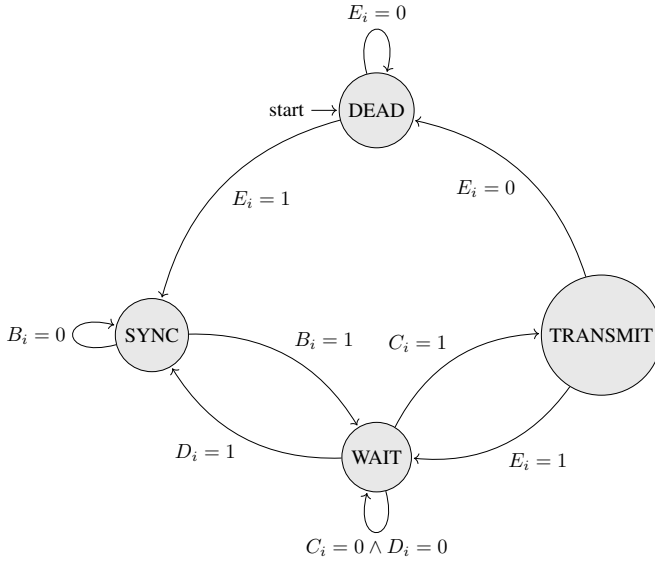


Fig. 4. State Diagram for Algorithm 2.

B. FSM for Algorithm 2

Fig. 4 illustrates the FSM for Algorithm 2. The FSM has four states: *DEAD*, *SYNC*, *WAIT*, and *TRANSMIT*, which implement phases *DEAD*, *SYNC*, *WAIT*, and *TRANSMIT* of Algorithm 2, respectively. Upon start-up, cluster member i enters state *DEAD*, and remains in this state until sufficient energy is harvested. When cluster member i has harvested sufficient energy, i.e., $E_i = 1$, it will transition to state *SYNC*, where it will wait for a synchronization packet. Once a synchronization packet is received, i.e., $B_i = 1$, the cluster member will transition to state *WAIT*, where it will wait for its designated time slot to transmit its data packet. While waiting, if a Sync1 or Sync2 packet is received, i.e., $D_i = 1$, cluster member i will transition back to state *SYNC*. Once the designated time slot is reached, i.e., $C_i = 1$, cluster member i will immediately transition to state *TRANSMIT*, where it will transmit its data packet and then check its remaining energy. If sufficient energy remained, i.e., $E_i = 1$, cluster member i will transition to state *WAIT*, otherwise, it will transition to state *DEAD*, continuing the same process from these two states.

Cluster member i waits for its designated time slot by setting a waiting time τ_w whenever it enters state *WAIT*. The value of τ_w is set depending on the prior state of state *WAIT* and the type of the synchronization packet. If the prior state is *SYNC* and the synchronization packet is Sync1, then the waiting time will be set to $\tau_w = \tau_n + (j - 1)t_s$, where τ_n is the current time moment when cluster member i enters state *WAIT* and $j \in \{1, 2, \dots, N\}$ is the time slot number designated to cluster member i . If the prior state is *SYNC* and the synchronization packet is a node packet or Sync2 packet, then the waiting time will be set to $\tau_w = \tau_n + ((N + j - k) \bmod N)t_s$, where k is the current running time slot included in the received packet. If the prior state is *TRANSMIT*, then the waiting time will be set to $\tau_w = \tau_n + Nt_s$, regardless of the synchronization packet. When τ_w expires, i.e., $\tau_w > \tau_c$, where τ_c is the current time, the cluster member transitions to state *TRANSMIT* to transmit its packet and then transitions back to state *WAIT*.

C. FSM for Algorithm 3

The FMS for Algorithm 3 is the same as the FSM of Algorithm 2 illustrated in Fig. 4. The difference is that while in states *SYNC* and *WAIT*, when receiving packets from its respective cluster members, the cluster head marks their timestamps and stores them. When transmitting a packet to the base station in state *TRANSMIT*, the cluster head includes its clusters members' packets and timestamps in its packet. The cluster head sets its waiting time similar to a cluster member.

VI. EXPERIMENTAL RESULTS

A. Experimental Setup

We conducted experiments on three different network setups: Setup 1, Setup 2, and Setup 3. Setup 1 comprises four nodes; one serves as a base station and three serve as cluster heads. Setup 2 also comprises four nodes; one node serves as a base station, one serves as a cluster head, and two serve as cluster members. Setup 3 has six nodes that are partitioned into two clusters; one serves as a base station, two serve as cluster heads (one for each cluster), and three serve as cluster members (two for the first cluster and one for the other cluster).

In each network setup, each node comprises an Arduino Uno R3 and a Digi XBee S2C. Arduino Xbee shields were used to affix the Digi XBee modules to the Arduino boards. Similar to [10], for all experiments, the energy availability probabilities of nodes are set to be the same, i.e., $p_i = p$, $\forall i \in \{1, 2, \dots, N\}$, and the time length of Sync packets is set to $52\mu s$. The data packet transmission time duration is $M_t = 70ms$.

For performance analysis, a separate XBee module, linked to a PC's serial port, was utilized to gather data from the networks. A Python script was developed to time, count, and retrieve all received data packets from this XBee module, saving the data to a text file in a comma-delimited format. Each entry in the text file represents a received packet, containing the reception timestamp, packet number, and the packet's data. The collected data was utilized to compute

the average network synchronization interval $\overline{T}_n$, the average cluster synchronization interval $\overline{T}_c$, and the average number of active phases $\overline{P}$.

B. Verification

To verify that the proposed algorithm works as expected, we first run the algorithm on Setup 1 when $p = 1$, which means that the nodes will never become out-of-energy, and thus all synchronization events will occur only due to cluster heads transmission overlap errors. We run the algorithm for different values of t_s , and for each value of t_s , we ran 4 trials; each trail consists of 20000 packets. For each trial, we calculated the average network synchronization interval $\overline{T}_n$ /trial, and then averaged the four trails to find the average network synchronization interval $\overline{T}_n$. Table I presents the values of $\overline{T}_n$ /trial and the values of $\overline{T}_n$ corresponding to each value of t_s . As anticipated, $\overline{T}_n$ increases as t_s increases. We repeated the same experiment on Setup 2, but in this experiment, we found the average cluster synchronization interval $\overline{T}_c$ instead of $\overline{T}_n$. We found that the values of $\overline{T}_c$ for each t_s value are almost the same as the $\overline{T}_n$ values found in Setup 1, which is expected as the TDAM structure of the two setups is the same.

TABLE I
AVERAGE SYNCHRONIZATION INTERVAL $\overline{T}_n$ FOR SETUP 1 AND $p = 1$.

$\overline{T}_n$ /trial (s)	Time Slot Length t_s (ms)						
	100	500	1000	1500	2000	2500	3000
Trial 1	33.04	461.19	1092.78	1671.90	2262.57	2803.75	3356.86
Trial 2	28.96	484.42	1093.41	1673.79	2253.21	2815.23	3361.85
Trial 3	26.78	497.32	1092.94	1670.93	2251.61	2816.08	3348.96
Trial 4	29.78	462.48	1094.60	1672.98	2252.63	2828.92	3440.94
$\overline{T}_n$	29.64	472.80	1093.10	1672.84	2257.89	2816.00	3377.15

TABLE II
AVERAGE NUMBER OF ACTIVE PHASES $\overline{P}$ FOR SETUP 2 AND $p = 0.8$.

$\overline{P}$ /trial	Time Slot Length t_s (ms)			
	100	500	1000	1500
Trial 1	127.27	135.61	120.43	132.63
Trial 2	122.34	121.13	126.32	131.29
Trial 3	123.59	131.97	143.99	124.36
Trial 4	129.16	126.32	114.93	130.76
Average	125.59	128.76	126.42	128.31
$\overline{P}$	127.63			

We repeated the same experiment on Setup 2 for $p = 0.8$ and $p = 0.4$. In this experiment, we computed the average number of active phases $\overline{P}$. Tables II and III outline the values of $\overline{P}$ /trial and $\overline{P}$ to each t_s value. As can be seen from the tables, $\overline{P}$ is 127.63 and 4.80 when $p = 0.8$ and $p = 0.4$, respectively. These $\overline{P}$ values are close to the theoretical values 125 and 4.63, respectively, calculated by the equation $1/(1-p)^N$, which is explained in [10].

We also run the proposed algorithm on Setup 3 when $p = 1$, and for three different values of t_s , which are 100ms, 500ms, and 1000ms. Three trials of 20000 packets were conducted for each value of t_s . Table IV outlines the average network

TABLE III
AVERAGE NUMBER OF ACTIVE PHASES $\overline{P}$ FOR SETUP 2 AND $p = 0.4$.

$\overline{P}$ /trial	Time Slot Length t_s (ms)			
	100	500	1000	1500
Trial 1	4.65	4.44	4.75	4.97
Trial 2	5.06	5.17	4.86	4.49
Trial 3	4.76	4.99	4.96	4.71
Trial 4	4.84	4.51	4.56	5.10
Average	4.83	4.78	4.78	4.82
$\overline{P}$	4.80			

TABLE IV
AVERAGE SYNCHRONIZATION INTERVAL $\overline{T}_n$ FOR SETUP 3 AND $p = 1$.

$\overline{T}_n$ /trial	Time Slot Length t_s (ms)		
	100	500	1000
Trial 1	4.59	447.61	801.63
Trial 2	4.87	379.56	860.56
Trial 3	4.25	505.16	882.90
$\overline{T}_n$	4.57	410.44	848.33

synchronization interval $\overline{T}_n$. As expected, $\overline{T}_n$ increases as t_s increases. Moreover, the values of $\overline{T}_n$ are smaller than the values of $\overline{T}_n$ for the single-hop, shown in Table I, which is also expected because Setup 1 is a single-hop, and thus nodes resynchronization occurs only due to one error, which is cluster heads transmission overlap error. Setup 3 is dual-hop, and thus nodes resynchronization occurs due two errors, which are cluster heads transmission overlap error and cluster members transmission overlap error. Two errors require nodes to resynchronize more often, which makes $\overline{T}_n$ of Setup 3 smaller than $\overline{T}_n$ of Setup 1.

C. Performance Evaluation

To evaluate the proposed algorithm, we compared its average energy consumption and average delay with the average energy consumption and average delay of the AT-MAC [8] and EH-TDMA [9] algorithms, and the algorithm in [10]. These algorithms are single-hop algorithms. Therefore, we used Setup 1 for comparison. For fair comparison, the algorithms were compared when $p = 1$. Moreover, the time length of the Sync packet in [10], the time length of the ping message in [9], the time length of the information packet in [8], the time length of the schedule packet in [8], and the time length of the beacon packet in [8] were set equal to the time length of the Sync packets in the proposed algorithm. As in [10], the transmit and receive powers are set to 1W and the time length of the acknowledgment packet for the algorithm in [10] is set to $24\mu s$.

The average energy consumption and average delay for the AT-MAC and EH-TDMA algorithms and the algorithm in [10] were calculated as described in [10], but when $t_s = 100ms$, which was found experimentally to be the minimum length

for t_s . The calculation of the average energy consumption and average delay of the proposed algorithm is similar to the calculation of the average energy consumption and average delay of [10], but without acknowledgment packets. Thus, in the proposed algorithm, the average energy consumption of a sensor node per $\overline{T}_n$ is equal to the energy consumption of receiving a Sync packet while the average energy consumption of the base station per $\overline{T}_n$ is equal to the energy consumption of transmitting $N+1$ Sync packets. The average delay is equal to the time length of one Sync packet.

TABLE V
AVERAGE ENERGY CONSUMPTION AND AVERAGE DELAY FOR SETUP 1.

	AT-MAC	EH-TDMA	[10]	Proposed
Energy per node (mJ)	10.32	5.14	0.076	0.052
Energy per coordinator (mJ)	20.6	5.14	0.28	0.21
Total Average Energy (mJ)	30.92	10.28	0.356	0.262
Delay (ms)	20.6	5.14	0.076	0.052

As can be seen from Table V, the proposed algorithm significantly reduces the average energy consumption and average delay compared to EH-TDMA and AT-MAC algorithms, which requires less energy harvesting rate. Compared to [10], the proposed algorithm reduces the average energy consumption by approximately 1.36 times and decreases the average delay by approximately 1.46 times. The proposed algorithm outperforms the algorithm in [10] because the proposed algorithm synchronizes the network without the need for acknowledgment packets.

VII. CONCLUSION

This paper proposes a TDAM algorithm and system for a dual-hop energy harvesting WSN, assuming that the network is partitioned into clusters. The proposed algorithm enables a sensor node to synchronize not only based on a synchronization packet from the base station, but also based on a data packet from another node that is within its communication range. The proposed algorithm enables individual synchronization of clusters and handles clock drift issues. Moreover, the proposed algorithm works for single-hop networks, and achieves network synchronization without the need for acknowledgment packets, resulting in lower average energy consumption and less average delay compared to the most recent single-hop algorithm. One future direction is extending the proposed algorithm to multi-hop scenarios.

REFERENCES

- [1] F. Alawad and F. A. Kraemer, "Value of information in wireless sensor network applications and the iot: A review," *IEEE Sensors Journal*, vol. 22, no. 10, pp. 9228–9245, 2022.
- [2] A. Ammar and D. Reynolds, "Energy harvesting networks: Energy versus data cooperation," in *IEEE Communications Letters*, vol. 22, no. 10, 2018, pp. 2128–2131.
- [3] A. M. A. Ammar, Y. P. Fallah, and D. Reynolds, "Throughput in an energy harvesting wireless uplink," *IEEE Sensors Journal*, vol. 18, no. 6, pp. 2617–2626, 2018.
- [4] R. Cui, Z. Qu, and S. Yin, "Energy-efficient routing protocol for energy harvesting wireless sensor network," in *15th IEEE International Conference on Communication Technology*, 2013, pp. 500–504.
- [5] Y. Luo, L. Pu, and Z. Peng, "Energy stimulated time synchronization for energy harvesting wireless networks," *IEEE Transactions on Network Science and Engineering*, vol. 9, no. 3, pp. 1880–1894, 2022.
- [6] Z. A. Eu, H.-P. Tan, and W. K. Seah, "Design and performance analysis of mac schemes for wireless sensor networks powered by ambient energy harvesting," *Ad Hoc Networks*, vol. 9, no. 3, pp. 300–323, 2011.
- [7] J. Xu, J. Kan, and Y. Zhang, "Centralized energy harvesting-based tdma protocol for terahertz nanosensor networks," *Sensors*, vol. 19, no. 20, 2019.
- [8] X. Qi, K. Wang, D. Yue, L. Shu, Y. Liu, and H. Zhao, "Adaptive tdma-based mac protocol in energy harvesting wireless body area network for mobile health," in *41st Annual Conference of the IEEE Industrial Electronics Society*, 2015, pp. 004 175–004 180.
- [9] S. Kosunalp, "Eh-tdma: A tdma-based mac protocol for energy-harvesting wireless sensor networks," *International Journal of Computer Science and Information Security*, vol. 14, pp. 325–328, 08 2016.
- [10] R. J. Hayek and A. Ammar, "An energy-efficient tdma algorithm for energy harvesting wireless sensor networks," in *2023 IEEE 14th Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON)*, 2023, pp. 0333–0338.
- [11] D. Hauweele and B. Quoitin, "Toward accurate clock drift modeling in wireless sensor networks simulation," *Computer Communications*, vol. 163, pp. 1–11, 2020.
- [12] R. Tjoa, K. Chee, P. Sivaprasad, S. Rao, and J. Lim, "Clock drift reduction for relative time slot tdma-based sensor networks," in *IEEE 15th International Symposium on Personal, Indoor and Mobile Radio Communications*, vol. 2, 2004, pp. 1042–1047 Vol.2.
- [13] Pei-Shan Yu, Jemin Lee, Tony Q. S. Quek, and Y.-W. Peter Hong, "Energy harvesting personal cells-traffic offloading and network throughput," in *IEEE International Conference on Communications (ICC)*, 2015, pp. 2184–2189.